

Codebase Cartographer - Comprehensive Analysis Report

Overview

Application Name: Codebase Cartographer
Description: An AI-powered system behavior mapping and data flow cartography tool for analyzing codebases.
Source: Exported from Google AI Studio

Tech Stack & Framework

Core Technologies

Technology	Version	Purpose
React	19.2.4	UI Framework
TypeScript	~5.8.2	Type-safe JavaScript
Vite	6.2.0	Build tool & dev server
D3.js	7.9.0	Data visualization (flow maps)
Tailwind CSS	CDN	Styling (loaded via script tag)
@google/genai	^1.40.0	Google Gemini API client

Build Configuration

- **Dev Server Port:** 3000 (configurable in vite.config.ts)
- **Module System:** ES Modules
- **Target:** ES2022

Project Structure

```

codebase-cartographer/
├── index.html           # Entry HTML with importmap for ESM dependencies
├── index.tsx           # React root mount point
├── App.tsx             # Main application component
├── types.ts            # TypeScript type definitions
├── constants.ts        # App constants and system prompts
├── vite.config.ts      # Vite build configuration
├── tsconfig.json       # TypeScript configuration
├── package.json        # NPM dependencies
├── .env.local          # Environment variables (API keys)
├── .gitignore          # Git ignore rules
├── metadata.json       # App metadata (name, permissions)
├── README.md           # Documentation
├── services/
│   ├── geminiService.ts # ALL LLM integration logic
│   └── ingestService.ts  # Codebase ingestion (GitHub/local)
├── components/
│   ├── LiveSession.tsx  # Real-time audio chat with AI
│   ├── FlowMap.tsx      # D3.js visualization component
│   ├── AssetGenerator.tsx # Video generation component
│   └── RepoIngest.tsx    # Repository ingestion modal

```

Application Core Functionality

What “Cartography” Means in This Context

The term “cartography” refers to **mapping the architecture and data flows of codebases**. Just as a cartographer creates maps of geographical terrain, this tool creates visual and analytical maps of:

- Code architecture
- Data flows
- Entry points (CLI, API, UI)
- LLM/AI integration points
- Configuration-driven behaviors

Main Purpose

This application helps developers:

1. **Analyze codebases** by ingesting file structures from local directories or GitHub
2. **Trace data flows** through systems via AI-powered analysis
3. **Visualize architectures** using interactive node-link graphs
4. **Generate documentation assets** including AI-generated videos
5. **Have conversations** with an AI expert about system behavior

Key Features

Feature	Description	Component
Chat Interface	Converse with AI about codebase architecture	<code>App.tsx</code>
Visual Flow Map	Interactive D3.js graph visualization	<code>FlowMap.tsx</code>
Codebase Ingestion	Load repos from GitHub or local filesystem	<code>RepoIngest.tsx</code>
Live Audio Session	Real-time voice interaction with AI	<code>LiveSession.tsx</code>
Asset Generation	AI-generated video walk-throughs	<code>AssetGenerator.tsx</code>
Thinking Mode	Extended AI reasoning (16k token budget)	Toggle in sidebar
Live Grounding	Google Search integration for real-time info	Toggle in sidebar
Auto-Map	Automatically updates visualization from chat	Enabled by default

LLM Integration Points (CRITICAL)

Current Provider: Google Gemini (Exclusively)

All LLM functionality is provided by Google Gemini through the `@google/genai` SDK.

API Key Configuration

Current Location: `.env.local`

```
GEMINI_API_KEY=PLACEHOLDER_API_KEY
```

How it's loaded: Via Vite's `loadEnv()` in `vite.config.ts`:

```
define: {  
  'process.env.API_KEY': JSON.stringify(env.GEMINI_API_KEY),  
  'process.env.GEMINI_API_KEY': JSON.stringify(env.GEMINI_API_KEY)  
}
```

Usage in code: `process.env.API_KEY` (single key for all operations)

Model Types Defined (types.ts)

Model ID	Constant Name	Purpose
gemini-3-pro-preview	SMART_THINKING	Complex reasoning with thinking mode
gemini-3-flash-preview	FAST_CHAT	Quick responses, search grounding
gemini-2.5-flash-native-audio-preview-12-2025	LIVE_AUDIO	Real-time audio streaming
gemini-2.5-flash-preview-tts	TTS	Text-to-speech generation
gemini-3-pro-image-preview	IMAGE_GEN	Image generation (not actively used)
veo-3.1-fast-generate-preview	VIDEO_GEN	Video generation

LLM Integration Summary Table

Function	File	Model Used	Task	Input	Output
generateTextResponse()	geminiService.ts	SMART_THINKING or FAST_CHAT	Chat/reasoning about codebases	Text history, optional image	Text response, grounding URLs
generateGraphData()	geminiService.ts	FAST_CHAT	Convert descriptions to graph JSON	Text description	JSON {nodes, links}
generateSpeech()	geminiService.ts	TTS	Text-to-speech for messages	Text	Base64 audio
generateVideo()	geminiService.ts	VIDEO_GEN (Veo)	Generate video walk-throughs	Prompt + optional image	Video blob URL
ai.live.connect()	LiveSession.tsx	LIVE_AUDIO	Real-time bi-directional audio	Microphone stream	Audio response

Detailed LLM Function Analysis

1. `generateTextResponse()` - Primary Chat Function

Location: `services/geminiService.ts:8-69`

Capabilities:

- Multi-turn conversation with history
- Optional “Thinking Mode” (16k token budget for deep reasoning)
- Optional Google Search grounding
- Image input support (multimodal)
- Returns grounding URLs when search is enabled

Model Selection Logic:

```
const modelId = useThinking ? ModelType.SMART_THINKING : ModelType.FAST_CHAT;
```

2. `generateGraphData()` - Structured Output

Location: `services/geminiService.ts:71-115`

Capabilities:

- Converts textual system descriptions to graph JSON
- Uses JSON schema for structured output
- Always uses `FAST_CHAT` model

3. `generateSpeech()` - Text-to-Speech

Location: `services/geminiService.ts:117-134`

Capabilities:

- Converts text to spoken audio
- Uses “Kore” voice preset
- Returns base64-encoded audio

4. `generateVideo()` - Video Generation (Veo)

Location: `services/geminiService.ts:137-189`

Capabilities:

- Generates videos from text prompts
- Optional image input for image-to-video
- Polling for async video generation
- Returns blob URL for playback

Special Note: Includes AI Studio integration for key selection:

```
if (window.aistudio && window.aistudio.hasSelectedApiKey) { ... }
```

5. Real-time Audio (Live Session)

Location: `components/LiveSession.tsx:24-140`

Capabilities:

- Bidirectional audio streaming
 - PCM audio encoding/decoding
 - Uses “Zephyr” voice preset
 - System instruction for cartographer persona
-

System Prompt / Instructions

Location: `constants.ts`

The `CARTOGRAPHER_SYSTEM_INSTRUCTION` defines the AI persona as:

“The world’s leading expert in data flow cartography and system behavior mapping for complex codebases”

Methodology phases:

- 1. **PHASE 1:** Initial Repo Reconnaissance
- 2. **PHASE 2:** Deep Data Flow Tracing
- 3. **PHASE 3:** Interactive Optimization & Guidance

Tone: Direct, evidence-backed, proactive, educational

Existing Configuration Systems

1. Environment Variables

- **File:** `.env.local`
- **Variables:** `GEMINI_API_KEY`
- **Loading:** Via Vite at build time

2. In-App Toggles (Runtime)

Toggle	State Variable	Effect
Thinking Mode	<code>useThinking</code>	Switches model, enables 16k thinking budget
Live Grounding	<code>useSearch</code>	Enables Google Search tool
Auto Map	<code>autoMap</code>	Auto-updates graph from chat responses

3. Model Type Enum

Centralized in `types.ts` - single source of truth for model identifiers.

4. Vite Config

`vite.config.ts` handles:

- Dev server configuration
- Environment variable injection
- Path aliases

Dependencies

Production Dependencies

```
{
  "@google/genai": "^1.40.0", // Google Gemini SDK
  "react": "^19.2.4",         // UI framework
  "react-dom": "^19.2.4",     // React DOM renderer
  "d3": "^7.9.0"              // Data visualization
}
```

Development Dependencies

```
{
  "@types/node": "^22.14.0",
  "@vitejs/plugin-react": "^5.0.0",
  "typescript": "~5.8.2",
  "vite": "^6.2.0"
}
```

How to Run the Application

1. Install dependencies:

```
bash
npm install
```

2. Set API key in `.env.local` :

```
GEMINI_API_KEY=your_actual_key_here
```

3. Start dev server:

```
bash
npm run dev
Opens at http://localhost:3000
```

4. Build for production:

```
bash
npm run build
```

Key Findings for Multi-LLM Support Implementation

Current Limitations

1. **Single API Key:** One `GEMINI_API_KEY` for all operations
2. **Single Provider:** Only Google Gemini supported
3. **Hardcoded Models:** Model IDs are constants, not configurable
4. **No Key Management UI:** No way to input keys at runtime
5. **No Capability Checking:** No warnings when a model can't perform a task

Files Requiring Modification

- 1. `types.ts` - Add provider types, capability definitions
- 2. `services/geminiService.ts` - Abstract to support multiple providers
- 3. `vite.config.ts` - May need multiple env vars
- 4. `App.tsx` - Add settings UI for API key management
- 5. **New file needed:** Configuration/settings component
- 6. **New file needed:** Provider abstraction layer

Tasks Each LLM Call Performs

Task Category	Current Function	Capabilities Needed
Text Generation	<code>generateTextResponse</code>	Chat, reasoning, grounding
Structured Output	<code>generateGraphData</code>	JSON generation
Text-to-Speech	<code>generateSpeech</code>	Audio output
Video Generation	<code>generateVideo</code>	Video output (Veo-specific)
Live Audio	<code>ai.live.connect</code>	Real-time audio I/O

Capability Matrix to Implement

Different LLMs have different capabilities. A configuration system should track:

-  Text generation
-  Chat/conversation
-  Structured JSON output
-  Image understanding (multimodal input)
-  Text-to-speech
-  Video generation (Veo-specific)
-  Real-time audio streaming (Gemini-specific)

Summary

Codebase Cartographer is a React + TypeScript application that uses Google Gemini for AI-powered codebase analysis and visualization. It currently relies on a single API key for all Gemini models. To support multiple LLM providers with user-configurable API keys, significant refactoring of the service layer will be needed, along with a new settings/configuration UI component.